THE UNIVERSITY OF SYDNEY

Due: March 27th, 2026 (Friday, Week 5) by 11:59 PM

COMP3221
Assignment 1: Routing Algorithms

Contents

1 Objectives **3**

2 Network Architecture **3**

3 Program Structure and Implementation Requirements **3**

3.1 Programming Language and Wrapper Script 3

3.2 Execution Format 4

4 Core Functions and Threading **4**

4.1 Listening Thread 4

4.2 Sending Thread 5

4.3 Routing Calculations Thread 5

4.4 Implementation Clarifications 6

5 Dynamic Commands and Graph Operations **7**

5.1 CHANGE 7

5.2 FAIL and RECOVER 7

5.3 QUERY 7

5.4 QUERY PATH 8

5.5 RESET 8

5.6 BATCH UPDATE 9

6 Optional Bonus Commands **9**

6.1 MERGE 9

6.2 SPLIT 10

6.3 CYCLE DETECT 10

7 Submission Requirements **10**
7.1 Files to be Submitted 10
7.2 Test Cases and Marking 11

1 Objectives

In this assignment, you are required to design and implement an advanced routing protocol that:

1. Computes least-cost paths using an algorithm of your choosing (e.g. Dijkstra, Bellman-Ford, etc.).
2. Handles dynamic changes in network topology.
3. Processes a variety of dynamic commands that modify the network state.
4. Performs advanced graph operations including detecting disjoint subgraphs, merging and splitting components, cycle detection and batch processing of commands.
5. Implements rigorous error handling for malformed inputs.

2 Network Architecture

The network is modeled as a **simple, undirected weighted graph** $G = (V, E)$, where each node $v \in V$ is identified by a unique identifier (e.g. A, B, C, ...), and each edge $(u, v) \in E$ has a non-negative cost $w(u, v) \in \mathbb{R}_{\geq 0}$. "Simple" means there are no self-loops and no parallel edges. Each node only knows about its immediate neighbours *initially*. Over time, via the routing protocol you implement, a node may learn about other nodes and edges in the network.

3 Program Structure and Implementation Requirements

3.1 Programming Language and Wrapper Script

You may implement your solution in any programming language supported by the submission system. However, your submission must include a wrapper script named `Routing.sh` that serves as the single entry point. For example, if you choose Python, your wrapper script might look as follows:

```
#!/bin/bash
python3 your_solution.py "$@"
```

You are *not* permitted to use any libraries outside the core libraries provided for your programming language. For example, if you were to use Python, you may use libraries such as `socket`, but not `networkx`. Using any disallowed libraries will result in an immediate 0 for this assignment. If you are unsure if a specific library is allowed, please ask on Ed.

3.2 Execution Format

Your program must be executed using the following command:

```
./Routing.sh <Node-ID> <Port-NO> <Node-Config-File> <RoutingDelay> <UpdateInterval>
```

where:

1. **Node-ID** is a unique identifier (e.g. A, B, C, ...).
2. **Port-NO** is the port on which the node listens (ports start at 6000 and increment by one).
3. **Node-Config-File** is a file specifying the node's immediate neighbours. The first line contains an integer n (the number of neighbours), followed by n lines. Each line contains three tokens: a neighbour's Node-ID, the cost (a non-negative floating-point number), and the neighbour's listening port.
4. **RoutingDelay** is the delay (in seconds) before the Routing Calculations Thread computes and outputs the routing table at startup.
5. **UpdateInterval** is the interval (in seconds) at which the Sending Thread broadcasts the current update packet via STDOUT.

For example, a configuration file for Node F (named `Fconfig.txt`) might be:

```
4
A 2.3 6000
C 3.2 6002
E 2.8 6004
J 5.4 6009
```

And the program could be executed as:

```
./Routing.sh F 6005 Fconfig.txt 60 10
```

4 Core Functions and Threading

Your program must use multi-threading and socket programming to handle the following tasks concurrently:

4.1 Listening Thread

The Listening Thread continuously reads messages from STDIN. It must process:

- **Update Packets:** in the exact format:

```
UPDATE <Source-Node> <Neighbour1>:<Cost1>:<Port1>,<Neighbour2>:<Cost2>:<Port2>,...
```

- **Dynamic Commands:** as described in Section 5.

Update packets must be forwarded to the Routing Calculations Thread, and dynamic commands must trigger immediate actions.

Communication Aspect: In addition to reading from STDIN, the Listening Thread must also handle incoming messages from other nodes via sockets.

4.2 Sending Thread

Every **UpdateInterval** seconds (as specified by the fifth command-line argument), the Sending Thread must broadcast the current update packet via STDOUT in the format, **only if there are changes since the last update**:

```
UPDATE <Node-ID>
    ↪ <Neighbour1>:<Cost1>:<Port1>,<Neighbour2>:<Cost2>:<Port2>,...
```

This output must exactly reflect the *current configuration of immediate neighbours* with no additional text.

Communication Aspect: Your solution must both print these update packets to STDOUT for logging/automated testing, and actively send them via sockets to your *neighbouring nodes* (i.e. only to the nodes with which you have a direct link). The actual messages you send over the socket *may contain any additional data or fields* to facilitate your routing algorithm, but the STDOUT format must match exactly.

4.3 Routing Calculations Thread

This thread is responsible for computing the least-cost paths:

- At startup (after a delay specified by the **RoutingDelay** command-line argument).
- Whenever a valid update packet or dynamic command is received **that changes the routing table.**

Immediately output the complete routing table in the format:

```
I am Node <Node-ID>
Least cost path from <Node-ID> to <Destination1>: <Path1>, link
    ↪ cost: <Cost1>
Least cost path from <Node-ID> to <Destination2>: <Path2>, link
    ↪ cost: <Cost2>
...
```

Note on Path Printing: Paths are printed as a continuous string of node IDs with no spaces. For example, if the least-cost path from A to D is A, B, C, D, then the output should be:

```
Least cost path from A to D: ABCD, link cost: <Cost>
```

4.4 Implementation Clarifications

Below are some key points regarding how you can implement the routing protocol and handle communication:

- **Update Packets and Neighbour Scope:** Each node's STDOUT `UPDATE` message must only list its *immediate neighbours* (the "1-hop" adjacency). You may, however, include additional fields or data structures in the socket-transmitted version of these packets if you want to implement a link-state approach.
- **Distance-Vector vs. Link-State:** You may choose any approach for computing shortest paths:
  - *Link-State*: Possibly sending more detailed info to allow each node to learn the entire topology and run the routing algorithm (e.g., Dijkstra) locally.
  - *Distance-Vector*: Sharing distance vectors with neighbours and converging (e.g., via Bellman-Ford style).

  The only strict requirement is that the `UPDATE` message to STDOUT must remain minimal (listing only your direct neighbours).
- **Constructing Full Paths vs. Next-Hop Tables:** The assignment requires each node to output full routes (e.g., "ABCD") in response to `QUERY` or `QUERY PATH`. You must maintain only next-hop info and reconstruct the path at query time by chaining next-hops.
- **Initial Knowledge vs. Learned Knowledge:** Each node starts with only the neighbours from its config file. Over time, it learns about other nodes/edges from `UPDATE` packets (and possibly dynamic commands). Your solution must gather enough information to answer `QUERY` and `QUERY PATH` for any reachable nodes.
- **Transport Protocol (TCP or UDP):** You may use either TCP or UDP for the node-to-node socket communication. Both are acceptable as long as you meet the functional requirements of the assignment.
- **Handling Dynamic Commands and Threads:** The specification describes three main threads (Listening, Sending, Routing Calculations). You may add extra threads if desired. You do not need to forward user commands (`FAIL`, `MERGE`, etc.) to other nodes; only broadcast the relevant changes if a command modifies your local routing or edge costs.
- **Key Points Summary:**
  - The `UPDATE` message *printed to STDOUT* must list *only* immediate neighbours.
  - Either distance-vector or link-state is acceptable.
  - Each node must learn about other nodes over time to answer queries.
  - Threading structure is flexible as long as concurrency is handled correctly.
  - You may store either a full graph or next-hop data as you prefer.
  - TCP or UDP are both acceptable.

5 Dynamic Commands and Graph Operations

Implement the following commands exactly as specified. Each command must produce the output or confirmation message described below.

5.1 CHANGE

Format:

```
CHANGE <NeighbourID> <NewCost>
```

Update the cost of the edge to `<NeighbourID>` with the new cost `<NewCost>`. Immediately broadcast the updated configuration (i.e. your new list of immediate neighbours and costs) and recalculate the routing table.

Example: If the command is

```
CHANGE B 3.5
```

then all paths using the edge to B must now use cost 3.5.

5.2 FAIL and RECOVER

Format:

```
FAIL <Node-ID>
RECOVER <Node-ID>
```

If `<Node-ID>` matches your node:

- On `FAIL`, stop broadcasting update packets and output:

  ```
  Node <Node-ID> is now DOWN.
  ```

- On `RECOVER`, resume broadcasting update packets and output:

  ```
  Node <Node-ID> is now UP.
  ```

If `<Node-ID>` does not match your node, simply update your routing table accordingly (e.g. treat that node as unreachable if it has failed).

Example: For the command

```
FAIL C
```

if your node is C, output `"Node C is now DOWN."`

5.3 QUERY

Format:

```
QUERY <Destination>
```

Output the least-cost path from your node to `<Destination>` in the format:

```
Least cost path from <Node-ID> to <Destination>: <Path>, link cost: <Cost>
```

Example: If your node is A and the command is

```
QUERY D
```

a possible output is:

```
Least cost path from A to D: ABCD, link cost: 7.2
```

5.4 QUERY PATH

Format:

```
QUERY PATH <Source> <Destination>
```

Output the least-cost path from `<Source>` to `<Destination>` (even if `<Source>` is not your node) in the same format as `QUERY`.

Example: For the command

```
QUERY PATH B E
```

the output could be:

```
Least cost path from B to E: BCE, link cost: 5.0
```

5.5 RESET

Format:

```
RESET
```

Reset your node's internal state and routing table to the configuration specified in the Node-Config-File. Immediately broadcast a new update packet and output:

```
Node <Node-ID> has been reset.
```

Then, output the routing table computed from the original configuration.

Example: For the command

```
RESET
```

if your node is A, output `"Node A has been reset."` followed by the routing table based on the original configuration.

5.6 BATCH UPDATE

Format:

```
BATCH UPDATE <Filename>
```

Read the file `<Filename>`, which contains one valid dynamic command per line (each command must exactly adhere to one of the formats specified above). Process these commands sequentially. After processing, output:

```
Batch update complete.
```

Then, recalculate and output the updated routing table.

Example: For the command

```
BATCH UPDATE updates.txt
```

output `"Batch update complete."` followed by the new routing table.

6 Optional Bonus Commands

In this section, we list extra commands that, if implemented correctly, are worth 20% additional points for this assignment. These commands are optional, as someone who did not attempt at implementing them can still get full mark at the assignment.

6.1 MERGE

Format:

```
MERGE <Node-ID1> <Node-ID2>
```

Merge the subgraph containing `<Node-ID2>` into the subgraph containing `<Node-ID1>`. After this command, the graph is updated so that all nodes formerly in the component of `<Node-ID2>` are now part of the component of `<Node-ID1>`. In cases of overlapping edges, select the edge with the lower cost. **Important:** The identity of `<Node-ID1>` remains unchanged; `<Node-ID2>` is absorbed into it. After merging, output:

```
Graph merged successfully.
```

Then recalculate and output the updated routing table.

Note: You can guarantee that the `MERGE` command will only be run on nodes that are part of the same subgraph.

Example: For the command

```
MERGE A C
```

Node C is merged into node A; all edges incident to C become edges incident to A (using the lower cost if duplicates exist).

6.2 SPLIT

Format:

```
SPLIT
```

Partition the current graph $G = (V, E)$ using the following fixed rule:

1. Sort the set of nodes V in alphabetical order.
2. Let $k = \lfloor |V|/2 \rfloor$. Define V_1 as the first k nodes and $V_2 = V \setminus V_1$.
3. Remove all edges connecting nodes in V_1 with nodes in V_2.

Each node outputs the routing table corresponding to the partition it belongs to. First, output:

```
Graph partitioned successfully.
```

Note: You can guarantee that the `SPLIT` command will only be run on nodes that are part of the same subgraph.

Example: If $V = \{A, B, C, D, E\}$ (in order A, B, C, D, E), then $k = 2$ and $V_1 = \{A, B\}$ while $V_2 = \{C, D, E\}$. Remove all edges between V_1 and V_2; each node outputs the routing table for its partition.

6.3 CYCLE DETECT

Format:

```
CYCLE DETECT
```

Analyse the current graph $G = (V, E)$ using a cycle detection algorithm (e.g. depth-first search). If a cycle exists, output:

```
Cycle detected.
```

Otherwise, output:

```
No cycle found.
```

Example: For the command

```
CYCLE DETECT
```

output one of the two messages as appropriate.

7 Submission Requirements

7.1 Files to be Submitted

You must submit the following files via Ed:

1. Source Code (in the programming language of your choice)
2. README (which must include details about your coding environment, package versions and instructions for running your program. This *will not be marked*, and is only for reference if there are issues when your program is run against the test cases. However, if you do not submit this file, you forfeit the option to challenge the failure of any test cases upon submission of the assignment.)

7.2 Test Cases and Marking

Test cases are divided into:

- **Public Test Cases:** Both the input and output of each case is visible.
- **Hidden Test Cases:** The input and output of these test cases is hidden, but you can still see whether or not they pass.
- **Private Test Cases:** The input and output are hidden, and you will not be able to see if you pass these test cases until after the deadline.

Each test case carries a specific weight, and your overall mark is determined by the cumulative weighted score across all test cases. Your program will be tested only against the criteria specified in this assignment.

Academic Honesty and Plagiarism

By submitting your assignment via ED, you agree to abide by the University policies on academic honesty. All work must be your original work. If any part of your submission is not your own, you must immediately inform your tutor or lecturer. Refer to the policy slides from Week 1 for further details. The School of Computer Science may share your assignment with a plagiarism checking service.

From Semester 1 2025, you can use AI tools like Microsoft Copilot Chat for your assignments, unless your unit coordinator has prohibited the class from using it. For exams and tests, you will not be allowed to use AI unless your unit coordinator has expressly permitted it. Different units and assessments will have different rules.

At the same time, it's important to understand when use of AI is unethical, inappropriate, or breaches the University's rules about academic integrity. Submitting assessments that aren't your original work – including work produced by AI – may constitute a breach of academic integrity.

In assessable work, the unapproved use of automated writing tools or generative AI, or failing to acknowledge them, is currently considered to be a breach of academic integrity, and penalties may apply. If you're unsure about whether you can use AI, or how you can use them, it's important that you speak with your unit coordinator.

For general learning purposes, as opposed to assessments, you are permitted to use generative AI tools, as long as you follow all University policies including the Academic

Integrity Policy 2022, Acceptable Use of ICT Resources Policy 2019 and the Student Charter 2020.

Appendix A: I/O Requirements and Error Handling

Below are some errors your program may encounter and how they need to be handled. Any errors beyond those listed here will not be tested; if in doubt, please ask on Ed.

General I/O Errors

Scenario	Input / Condition	Expected Output
Missing Arguments	`./Routing.sh F 6005`	`Error: Insufficient arguments provided. Usage: ./Routing.sh <Node-ID> <Port-NO> <Node-Config-File>`
Invalid Node-ID	`./Routing.sh AB 6005 Fconfig.txt`	`Error: Invalid Node-ID.`
Invalid Port Number	`./Routing.sh F xyz Fconfig.txt`	`Error: Invalid Port number. Must be an integer.`
Config File Not Found	`./Routing.sh F 6005 missing.txt`	`Error: Configuration file missing.txt not found.`
Non-numeric Neighbour Count	`(in config file, first line: four)`	`Error: Invalid configuration file format. (First line must be an integer.)`
Malformed Neighbour Entry	`(in config file: A two 6000)`	`Error: Invalid configuration file format. (Each neighbour entry must have exactly three tokens; cost must be numeric.)`
Extra Tokens in Config File	`(in config file: A 2.3 6000 extra)`	`Error: Invalid configuration file format.`
Malformed CLI Command	`e.g. SET COST A B abc`	`Error: Invalid command format. Expected numeric cost value.`

Appendix B: Dynamic Command and Graph Operation Error Handling

Command Scenario	Input / Condition	Expected Error Output
Malformed Update Packet	Packet not starting with UPDATE (e.g. UPD8 <Source> ↪ A:2.3:6000)	Error: Invalid update ↪ packet format.
Malformed CHANGE Command	CHANGE A two (not exactly two tokens or non-numeric cost)	Error: Invalid command ↪ format. Expected ↪ numeric cost value.
Malformed FAIL Command	FAIL AB (invalid node ID)	Error: Invalid command ↪ format. Expected a ↪ valid Node-ID.
Malformed RECOVER Command	RECOVER ab (invalid node ID)	Error: Invalid command ↪ format. Expected a ↪ valid Node-ID.
Malformed QUERY Command	QUERY AB (destination invalid)	Error: Invalid command ↪ format. Expected a ↪ valid Destination.
Malformed QUERY PATH Command	QUERY PATH A b (not exactly three tokens or invalid IDs)	Error: Invalid command ↪ format. Expected two ↪ valid identifiers for ↪ Source and Destination.
Malformed MERGE Command	MERGE A b (invalid node IDs)	Error: Invalid command ↪ format. Expected two ↪ valid identifiers for ↪ MERGE.
Malformed SPLIT Command	SPLIT extra (extra tokens)	Error: Invalid command ↪ format. Expected ↪ exactly: SPLIT.
Malformed RESET Command	RESET now (extra tokens)	Error: Invalid command ↪ format. Expected ↪ exactly: RESET.
Malformed CYCLE DETECT Command	CYCLE DETECT extra (extra tokens)	Error: Invalid command ↪ format. Expected ↪ exactly: CYCLE DETECT.
Malformed BATCH UPDATE Command	BATCH UPDATE (not exactly two tokens or missing filename)	Error: Invalid command ↪ format. Expected: ↪ BATCH UPDATE ↪ <Filename>.

Extra Tokens in Dynamic Command	e.g. CHANGE A 2.5 extra	Error: Invalid command ↪ format. Expected ↪ exactly two tokens ↪ after CHANGE.
Missing Tokens in Dynamic Command	e.g. FAIL	Error: Invalid command ↪ format. Expected: FAIL ↪ <Node-ID>.

Note: All error messages must be written exactly as shown. If any such error is detected, your program must exit immediately with a non-zero exit code.

The ↪ character refers to places where a newline is shown in the specifications, but not in the actual output.